34. Concurrent Algorithm Drills

34. Concurrent Algorithm Drills

Phase: Phase 5 — Concurrency
Estimated time: 2 days
Slug: `34-concurrent-algorithms`

Learning objectives

- Parallelize map-reduce style workloads
- Compute safely with bounded concurrency
- Compare sequential vs parallel benchmarks
- Preserve determinism in tests

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Drill themes

- Parallel sum / histogram with shard-per-worker then merge
- Concurrent crawler with URL graph + visited set (mutex or owning map goroutine)
- Pipeline: read → parse → enrich → write

Rules

- Bound parallelism to GOMAXPROCS or a chosen limit
- No shared map writes without sync
- Tests must be deterministic (sort outputs)

Measure: sometimes parallel is slower due to coordination overhead — learn when.

Common mistakes (especially from Python instincts)

- Racing on shared maps
- More goroutines than work items blindly
- Non-deterministic test flakes ignored

Exercises

Work in `exercises/34-concurrent-algorithms/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/34-concurrent-algorithms/` until you have an honest failing attempt.

Self-check

1. When parallelization fails to help?
2. How to merge partial histograms?
3. Why sort in tests?

Further reading

- https://pkg.go.dev/sync